01 한눈에 보기

매장 장부에서 숫자를 찾는 직원

어느 매장과 어느 기간을 볼지 받으면, DB에서 필요한 방문 기록만 골라 원하는 기준으로 묶어서 돌려줍니다.

1. 조건 받기

서비스에서 매장 번호와 시작·끝 시각을 받습니다.

2. 기록 골라 세기

그 매장의 기록만 모아 고객, 요일, 시간, 월별로 셉니다.

3. 결과 돌려주기

계산 결과를 한 줄씩 담은 목록을 서비스에 돌려줍니다.

파일 위쪽과 조회 도구의 이름

```
public interface AnalyticsRepository extends Repository<Visit, Long> {
```

package / import	package는 코드의 소속입니다. import는 Visit, Query, Repository, Param, LocalDateTime, List를 짧은 이름으로 쓰게 합니다.
interface / extends	인터페이스는 어떤 일을 요청할 수 있는지 적은 약속입니다. Spring Data가 이 약속과 조회문을 보고 실제로 동작하는 객체를 준비합니다.
Repository<Visit, Long>	이 저장소는 방문 기록인 Visit을 대상으로 하고 방문 기록의 ID 타입은 Long입니다.
@Query / @Param	@Query는 실행할 조회문을 붙이는 이름표입니다. @Param은 자바에서 받은 값을 조회문의 같은 이름 자리에 연결합니다.
LocalDateTime / List	LocalDateTime은 날짜와 시각을 담는 타입입니다. List는 조회 결과 여러 줄을 담는 목록입니다.
Javadoc	역할·작성자·작성일·변경노트를 적은 설명입니다. @param은 받는 값, @return은 돌려주는 값에 대한 설명입니다.

02 조회 조건과 고객별 집계

누구의 방문을 몇 번 셀까?

첫 조회는 고객별 결제 합계와 방문 횟수를 구합니다. 네 조회 모두 매장과 시간 조건으로 필요한 기록만 고릅니다.

1. 자바에서 값을 받는 칸 세 개

```
List<CustomerTotals> summarizeByCustomer(@Param("storeId") long storeId,
        @Param("start") LocalDateTime start, @Param("end") LocalDateTime end);
```

storeId는 숫자(long)를 받는 매개변수입니다. start와 end는 날짜·시각 객체를 가리키는 매개변수입니다. 서비스가 호출할 때 보내는 매장 번호와 시각은 인수입니다. 예를 들어 @Param("storeId")는 받은 번호를 :storeId 자리에 연결합니다.

2. 같은 고객끼리 묶어 금액과 건수를 구해요

```
select sum(v.amount) as revenue, count(v.id) as visitCount
from Visit v
where v.customer.store.id = :storeId
  and v.visitedAt >= :start
  and v.visitedAt < :end
group by v.customer.id
```

from Visit v	방문 기록에서 읽습니다. v는 Visit을 짧게 부르는 이름입니다. 이 조회문은 Java 객체와 필드 이름을 쓰는 JPQL입니다.
where / and	방문 → 고객 → 매장을 따라가 매장 번호를 맞춥니다. 시작 시각은 포함하고(>=), 종료 시각은 제외합니다(<).
group by	같은 고객 번호끼리 한 묶음으로 만듭니다. 방문한 고객마다 결과가 한 줄씩 나옵니다.
sum / count / as	sum은 결제 금액을 더하고, count는 방문 기록을 셉니다. as는 결과에 이름표를 붙입니다.

예를 들어 10월 조회에 10월 1일 0시는 들어가고 11월 1일 0시는 빠집니다. 다른 매장의 10월 방문도 매장 조건에서 빠집니다.

방문 기록을 두 가지로 나눠 세요

같은 방문 기록도 요일별로 모으면 바쁜 요일을, 시간별로 모으면 바쁜 시간을 알 수 있습니다.

요일별 / countByWeekday()

```
select dayofweek(v.visited_at) as dayNumber, count(*) as visitCount
from visit v
join customer c on c.id = v.customer_id
where c.store_id = :storeId
  and v.visited_at >= :start
  and v.visited_at < :end
group by dayofweek(v.visited_at)
```

dayofweek()는 방문 시각에서 요일 번호를 구합니다. MySQL에서는 일요일이 1, 월요일이 2, 토요일이 7입니다. group by로 같은 요일을 묶고 count(*)로 방문 줄을 셉니다. 결과 한 줄에는 dayNumber와 visitCount가 들어갑니다.

시간대별 / countByHour()

```
select hour(v.visited_at) as visitHour, count(*) as visitCount
from visit v
join customer c on c.id = v.customer_id
where c.store_id = :storeId
  and v.visited_at >= :start
  and v.visited_at < :end
group by hour(v.visited_at)
```

hour()는 저장된 시각에서 시(0~23)만 꺼냅니다. 10시 10분과 10시 50분은 같은 10시 칸에 들어가 2건이 됩니다. 결과 한 줄에는 visitHour와 visitCount가 들어갑니다.

두 조회에 공통으로 나오는 부분

join customer c	방문 기록에 적힌 고객 번호로 고객 표의 같은 번호를 찾습니다. 고객이 소속된 매장 번호를 확인할 수 있습니다.
nativeQuery = true	Java 객체 이름 대신 DB의 visit, customer, visited_at 같은 실제 표·칸 이름을 쓰는 SQL임을 알려줍니다.

방문이 없는 요일이나 시간은 DB 결과에 줄이 없습니다. AnalyticsService가 빈 칸을 0으로 채웁니다.

월별 숫자를 결과 목록에 담아요

매출은 방문 기록의 결제 금액을 더하고, 방문 고객 수는 같은 사람이 여러 번 와도 한 명으로 셉니다.

월별 / summarizeByMonth()

```
select year(v.visited_at) as visitYear, month(v.visited_at) as visitMonth,
       sum(v.amount) as revenue, count(distinct v.customer_id) as customerCount
from visit v
join customer c on c.id = v.customer_id
where c.store_id = :storeId
  and v.visited_at >= :start
  and v.visited_at < :end
group by year(v.visited_at), month(v.visited_at)
```

year / month	방문 시각의 연도와 월을 꺼냅니다. 둘을 같이 묶어 2026년 10월과 2027년 10월을 구분합니다.
sum(v.amount)	그 달에 방문 기록에 적힌 결제 금액을 합칩니다.
count(distinct ...)	distinct는 같은 고객 번호를 한 번만 셉니다. 한 고객이 두 번 오면 방문 기록은 2건, 방문 고객은 1명입니다.

결과 인터페이스는 한 줄에 들어갈 값의 약속

CustomerTotals	getRevenue()로 고객의 결제 합계, getVisitCount()로 그 고객의 방문 건수를 읽습니다.
WeekdayTotal	getDayNumber()로 요일 번호, getVisitCount()로 그 요일의 방문 건수를 읽습니다.
HourlyTotal	getVisitHour()로 시간, getVisitCount()로 그 시간의 방문 건수를 읽습니다.
MonthlyTotal	getVisitYear(), getVisitMonth(), getRevenue(), getCustomerCount()로 연도, 월, 매출, 고유 방문 고객 수를 읽습니다.

as로 붙인 이름은 getRevenue()처럼 값을 꺼내는 메서드와 연결됩니다. 조회 메서드는 결과 객체 여러 줄을 List로 돌려주고, 기간에 해당하는 기록이 없으면 빈 목록을 돌려줍니다.